

1. Dalla proprietà del max-heap, ciascun nodo è $\geq$ dei suoi figli. Il più piccolo elemento sarà quindi una foglia e sappiamo che il suo indice i è
- $$\lfloor \frac{n}{2} \rfloor + 1 \leq i \leq n \quad (\text{vedi Lez 17}).$$

2. Sì, un array ordinato è un min-heap.
Infatti, se esistono, i figli dx e dx di un elemento con indice i hanno indici $2i$ & $2i+1$, rispettivamente, e quindi contengono valori $\geq$ di quello del padre (poiché l'array è ordinato).

3.

MIN-HEAPIFY (A, i)

$l = \text{LEFT}(i)$

$r = \text{RIGHT}(i)$

if $l \leq A.\text{heap-size}$ and $A[l] < A[i]$

$\text{smallest} = l$

else $\text{smallest} = i$

if $r \leq A.\text{heap-size}$ and $A[r] < A[\text{smallest}]$

$\text{smallest} = r$

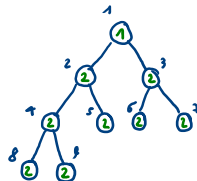
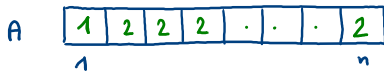
if $\text{smallest} \neq i$

 exchange $A[i]$ with $A[\text{smallest}]$

 MIN-HEAPIFY ($A, \text{smallest}$)

Tempo esecuzione: $O(\log n)$

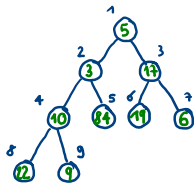
4. È sufficiente fornire un array A di n , dato in input a MAX-HEAPIFY richiede $\Theta(\log n)$.



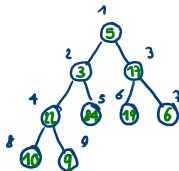
Chiamo MAX-HEAPIFY($A, 1$).

Viene eseguito un numero di swap pari all'altezza dell'heap, che è $\lfloor \log n \rfloor$.

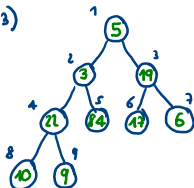
5.



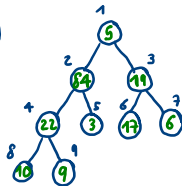
MAX-HEAPIFY(A, 4)



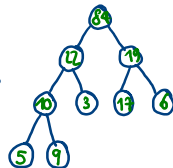
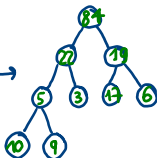
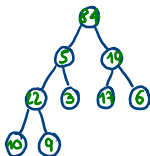
MAX-HEAPIFY(A, 3)



MAX-HEAPIFY(A, 2)



MAX-HEAPIFY(A, 1)



7.

Abbiamo visto nell'es. 1 che il minimo potrebbe essere una qualsiasi delle foglie, che sono $\lceil \frac{n}{2} \rceil$.

non abbiamo modo di sapere quale se non scorrendole tutte
 $\rightarrow \Theta(n)$ nel caso peggiore.